

Rajalakshmi Engineering College

Name: Divya S
Email: 241001056@rajalakshmi.edu.in
Roll no:
Phone: 9345901972
Branch: REC
Department: IT - Section 1
Batch: 2028
Degree: B.E - IT

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 7_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 14

Section 1 : MCQ

1. What is the output of the following code?

```
interface MathOperations {  
    static int square(int x) {  
        return x * x;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        System.out.println(MathOperations.square(5));  
    }  
}
```

Answer

25

Status : Correct

Marks : 1/1

2. Which of the following is the correct way to declare an interface in Java?

Answer

```
interface Vehicle { void start();}
```

Status : Correct

Marks : 1/1

3. What happens when an implementing class does not override a default method from an interface?

Answer

The default method's implementation from the interface will be used.

Status : Correct

Marks : 1/1

4. How can a class explicitly call a default method from an interface if there is a naming conflict?

Answer

Using InterfaceName.super.methodName();

Status : Correct

Marks : 1/1

5. What is the output of the following code?

```
interface X {  
    default void show() {  
        System.out.println("X's Default Method");  
    }  
}
```

```
interface Y {  
    default void show() {
```

```
        System.out.println("Y's Default Method");
    }
}
```

```
class Z implements X, Y {
    public void show() {
        System.out.println("Z's Method");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Z obj = new Z();
        obj.show();
    }
}
```

Answer

Z's Method

Status : Correct

Marks : 1/1

6. Which of the following statements about Java interfaces is true?

Answer

A class can implement multiple interfaces.

Status : Correct

Marks : 1/1

7. Consider a class implementing an interface and extending a class, both having a method with the same name. Which method gets called?

Answer

The method from the superclass

Status : Correct

Marks : 1/1

8. If a class implements two interfaces that have the same default

method, what must the class do?

Answer

The class must override the method to resolve ambiguity.

Status : Correct

Marks : 1/1

9. What is the output of the following code?

```
interface A {  
    default void show() {  
        System.out.println("A's Default Method");  
    }  
}  
  
class B {  
    public void show() {  
        System.out.println("B's Method");  
    }  
}  
  
class C extends B implements A {  
}  
  
public class Main {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.show();  
    }  
}
```

Answer

B's Method

Status : Correct

Marks : 1/1

10. What is the primary purpose of static methods in Java interfaces?

Answer

They allow an interface to provide helper methods without requiring an implementing class.

Status : Correct

Marks : 1/1

11. Can a Java interface contain both default and static methods?

Answer

Yes, an interface can have both default and static methods.

Status : Correct

Marks : 1/1

12. How do you call a static method from an interface MyInterface?

Answer

MyInterface.staticMethod();

Status : Correct

Marks : 1/1

13. What is the output of the following code?

```
interface A {  
    static void display() {  
        System.out.println("Static method in A");  
    }  
}
```

```
class B implements A {  
    static void display() {  
        System.out.println("Static method in B");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        B.display();  
    }  
}
```

```
}  
}
```

Answer

Compilation Error

Status : Wrong

Marks : 0/1

14. Which of the following statements is true regarding default methods in Java interfaces?

Answer

A default method can be overridden in a class implementing the interface.

Status : Correct

Marks : 1/1

15. What is the output of the following code?

```
interface A {  
    default void show() {  
        System.out.println("A's Default Method");  
    }  
}
```

```
interface B {  
    default void show() {  
        System.out.println("B's Default Method");  
    }  
}
```

```
class C implements A, B {  
    public void show() {  
        A.super.show();  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {
```

```
        C obj = new C();  
        obj.show();  
    }  
}
```

Answer

A's Default Method

Status : Correct

Marks : 1/1

Rajalakshmi Engineering College

Name: Divya S
Email: 241001056@rajalakshmi.edu.in
Roll no:
Phone: 9345901972
Branch: REC
Department: IT - Section 1
Batch: 2028
Degree: B.E - IT

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 7_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement:

Rajiv is analyzing the energy consumption in his household and wants to calculate the total cost based on the daily energy usage. He is given the rate per unit of electricity and the energy consumed for multiple days. To structure this calculation efficiently, he decides to use an interface-based approach.

Implement an interface CostCalculator with the necessary methods to retrieve energy details and compute the cost. The calculations should be handled in the EnergyConsumptionTracker class, while the EnergyConsumptionApp class should only handle input and output.

Formula

Energy Cost for one day = Energy Consumed per day * Rate Per Unit

Input Format

The first line of input consists of the rate per unit as an 'R' (a double value).

The second line of input consists of the number of days 'N' (an integer).

The third line of input consists of the daily energy consumption values for each day 'D' (double values), separated by space.

Output Format

The first line of the output prints: "Day-wise Energy Cost:"

The next N lines of the output print the day-wise energy costs(double type) and the total energy cost (double type) in Indian Rupees in the following format: "Day [day_number]: Rs. [energy_cost]"

The last line of the output prints: "Total Energy Cost: Rs. [total_cost]"

Note: energy_cost and total_cost are rounded off to two decimal points

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 0.01

3

10.0 20.0 30.0

Output: Day-wise Energy Cost:

Day 1: Rs. 0.10

Day 2: Rs. 0.20

Day 3: Rs. 0.30

Total Energy Cost: Rs. 0.60

Answer

```
import java.util.Scanner;
```

```
interface CostCalculator{
```

```
    void getEnergyDetails(Scanner scanner);  
    void calculateAndDisplayCost();
```

```
}
```

```
class EnergyConsumptionTracker implements CostCalculator
```

```
{
```

```
    private double ratePerUnit;  
    private int numDays;  
    private double[] dailyConsumption;
```

```
    public EnergyConsumptionTracker(double ratePerUnit, int numDays)
```

```
{
```

```
        this.ratePerUnit = ratePerUnit;  
        this.numDays = numDays;  
        this.dailyConsumption = new double[numDays];
```

```
}
```

```
    public void getEnergyDetails(Scanner scanner)
```

```
{
```

```
        for (int i = 0; i < numDays; i++)
```

```
{
```

```

        dailyConsumption[i] = scanner.nextDouble();

    }

}

public void calculateAndDisplayCost()
{

    double totalCost = 0.0;
    System.out.println("Day-wise Energy Cost:");
    for (int i = 0; i < numDays; i++)
    {

        double dayCost = dailyConsumption[i] * ratePerUnit;
        totalCost += dayCost;
        System.out.printf("Day %d: Rs. %.2f%n", (i + 1), dayCost);

    }

    System.out.printf("Total Energy Cost: Rs. %.2f", totalCost);

}

}

class EnergyConsumptionApp {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double ratePerUnit = scanner.nextDouble();
        int numDays = scanner.nextInt();

        CostCalculator tracker = new EnergyConsumptionTracker(ratePerUnit,
numDays);

```

```
        tracker.getEnergyDetails(scanner);  
        tracker.calculateAndDisplayCost();  
    scanner.close();  
    }  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Divya S
Email: 241001056@rajalakshmi.edu.in
Roll no:
Phone: 9345901972
Branch: REC
Department: IT - Section 1
Batch: 2028
Degree: B.E - IT

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 7_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Jaheer is working on a health monitoring system to help individuals calculate their Body Mass Index (BMI). He has implemented a basic BMI calculator and an interface called HealthCalculator. It should have a method called calculateBMI.

You are tasked with creating a program that takes weight and height as input, calculates the BMI using the BMICalculator class, and displays the result. If the height or weight is less than or equal to zero, then return -1.

Formula: $BMI = \text{weight} / (\text{height} * \text{height})$

Input Format

The first line of input consists of a double value W, the person's weight in kilograms.

The second line consists of a double value H, the height of the person in meters.

Output Format

The output displays "BMI: " followed by a double value, representing the calculated BMI, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 70.0

1.75

Output: BMI: 22.86

Answer

```
import java.util.Scanner;
```

```
interface HealthCalculator{
```

```
    double calculateBMI(double weight, double height);
```

```
}
```

```
class BMICalculator implements HealthCalculator
```

```
{
```

```
    public double calculateBMI(double weight, double height)
```

```
{
```

```
        if (weight <= 0 || height <= 0)
```

```

{

    return -1;

}

return weight / (height * height);

}

}

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double weight = scanner.nextDouble();
        double height = scanner.nextDouble();

        BMICalculator bmiCalculator = new BMICalculator();

        double bmi = bmiCalculator.calculateBMI(weight, height);

        System.out.printf("BMI: %.2f\n", bmi);

        scanner.close();
    }
}

```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Divya S
Email: 241001056@rajalakshmi.edu.in
Roll no:
Phone: 9345901972
Branch: REC
Department: IT - Section 1
Batch: 2028
Degree: B.E - IT

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 7_Q3

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

A financial analyst, Alex, needs a program to calculate simple interest for various financial transactions. He requires a straightforward tool that takes in the principal amount, interest rate, and time in years and computes the interest.

The formula to be used is: $\text{Interest} = \text{Principal} \times \text{Rate} \times \text{Time} / 100$

Implement this functionality using the InterestCalculator interface and the SimpleInterestCalculator class.

Input Format

The first line of input consists of the principal amount P as a double value.

The second line of input consists of the annual interest rate r as a double value.

The third line of input consists of the number of years t as a positive integer, which is an integer value.

Output Format

The output displays the calculated simple interest in the following format: "Simple Interest: [interest_value]", Here, [interest_value] should be replaced with the actual interest value calculated by the program.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1000.00

5.00

2

Output: Simple Interest: 100.0

Answer

```
import java.util.Scanner;
```

```
// You are using Java
```

```
interface InterestCalculator{
```

```
    double simpleInterest(double principal, double rate, int time);
```

```
}
```

```
class SimpleInterestCalculator implements InterestCalculator
```

```
{
```

```
    public double simpleInterest(double principal, double rate, int time)
```

```
{  
  
    return (principal * rate * time) / 100;  
  
}  
  
}  
  
class Main {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        double principal = scanner.nextDouble();  
  
        double rate = scanner.nextDouble();  
  
        int time = scanner.nextInt();  
  
        InterestCalculator calculator = new SimpleInterestCalculator();  
  
        double interest = calculator.simpleInterest(principal, rate, time);  
  
        System.out.println("Simple Interest: " + interest);  
  
    }  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Divya S
Email: 241001056@rajalakshmi.edu.in
Roll no:
Phone: 9345901972
Branch: REC
Department: IT - Section 1
Batch: 2028
Degree: B.E - IT

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 7_Q4

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Maria, a software developer, is working on an inventory management system project using Java that utilizes an inventory interface to manage a store's products.

The interface should define two methods: `addProduct`, which adds a product by accepting its name, price, and quantity, and `calculateTotalValue`, which computes the total value of all products in the inventory. Implement the interface in a class called `SimpleInventory`, which internally manages a list of `Product` objects.

Each `Product` object should encapsulate the product's name, price, and quantity and include a method to calculate its value as $\text{price} \times \text{quantity}$. The system should allow users to dynamically add products to the inventory and calculate the total value of all products stored.

Help Maria achieve the task.

Input Format

The first line of input consists of an integer to choose one of the following options:

- 1 - to add a product to the inventory.
- 2 - to calculate and view the total inventory value.
- 3 - to exit the program.

For Choice 1 (Add Product):

The next input line is the string representing the product name as a string (single or multi-word, without quotes).

The next line is a double value representing the price as a decimal value

The next line is an integer value representing the quantity as an integer

For Choices 2 and 3, no additional input is required

Output Format

The output displays the results of the commands as follows:

- For the addProduct command, the program should display "Product added to inventory."
- For choice 2, the program should display "Total inventory value [totalvalue].
"The total value should be displayed with one decimal place. If there is no product in the inventory, print the total as 0.0.
- For choice 3, the program should exit

If the choice is not 1, 2, or 3, then print "Invalid choice. Please select a valid option (1/2/3).".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1

Laptop

800.0

3

2

5

3

Output: Product added to inventory.

Total inventory value: \$2400.0

Invalid choice. Please select a valid option (1/2/3).

Answer

```
import java.util.Scanner;
```

```
// You are using Java
```

```
class Product{
```

```
    private String name;
```

```
    private double price;
```

```
    private int quantity;
```

```
    public Product(String name, double price, int quantity)
```

```
{
```

```
        this.name = name;
```

```
        this.price = price;
```

```
        this.quantity = quantity;
```

```
}
```

```
    public double calculateValue()
```

```
{
```

```
    return price * quantity;
```

```
}
```

```
}
```

```
interface Inventory
```

```
{
```

```
    void addProduct(String name, double price, int quantity);  
    double calculateTotalValue();
```

```
}
```

```
class SimpleInventory implements Inventory
```

```
{
```

```
    private Product[] products;  
    private int count;
```

```
    public SimpleInventory(int capacity)
```

```
{
```

```
        products = new Product[capacity];  
        count = 0;
```

```
}
```

```
    public void addProduct(String name, double price, int quantity)
```

```
{
```

```
        if (count < products.length)
        {

            products[count++] = new Product(name, price, quantity);
            System.out.println("Product added to inventory.");

        }

    }

    public double calculateTotalValue()
    {

        double total = 0.0;
        for (int i = 0; i < count; i++)
        {

            total += products[i].calculateValue();

        }

        return total;

    }

}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Inventory inventory = new SimpleInventory(10);
        while (true) {
```

```

int choice = scanner.nextInt();
if (choice == 1) {
    scanner.nextLine();
    String productName = scanner.nextLine();
    double price = scanner.nextDouble();
    int quantity = scanner.nextInt();
    inventory.addProduct(productName, price, quantity);
} else if (choice == 2) {
    double totalValue = inventory.calculateTotalValue();
    System.out.println("Total inventory value: $" + totalValue);
} else if (choice == 3) {
    break;
} else {
    System.out.println("Invalid choice. Please select a valid option
(1/2/3).");
}
}
scanner.close();
}
}

```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Divya S
Email: 241001056@rajalakshmi.edu.in
Roll no:
Phone: 9345901972
Branch: REC
Department: IT - Section 1
Batch: 2028
Degree: B.E - IT

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 7_Q5

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Raj is curious about how old he is in the current year.

He has asked you to create a simple program that calculates a person's age based on their birth year. You decide to implement this functionality using the AgeCalculator interface and the HumanAgeCalculator class.

Note: The current year is 2024. Calculate the current age by using the formula: current year - birth year.

Input Format

The input consists of an integer representing the birth year.

Output Format

The output displays "You are X years old." where X is an integer representing the calculated age based on the entered birth year.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1934

Output: You are 90 years old.

Answer

```
import java.util.Scanner;

// You are using Java
interface AgeCalculator{
    int calculateAge(int birthYear);
}
class HumanAgeCalculator implements AgeCalculator{
    public int calculateAge(int birthYear){
        int currentYear=2024;
        return currentYear-birthYear;
    }
}

class AgeCalculatorApp {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        AgeCalculator ageCalculator = new HumanAgeCalculator();

        int birthYear = scanner.nextInt();
        int age = ageCalculator.calculateAge(birthYear);

        System.out.println("You are " + age + " years old.");
    }
}
```

Status : Correct

Marks : 10/10